Linear Algebra Roadmap for Machine Learning

Contents

1. Overview . . . 2
2. Why ML needs linear algebra . . . 3
   - 2.1 A neural network is matrix maths . . . 3
   - 2.2 Two ways to know linear algebra . . . 5
3. How to read the roadmap . . . 5
4. The eight modules . . . 6
   - 4.1 Scalars . . . 7
   - 4.2 Vectors . . . 7
   - 4.3 Matrices . . . 8
   - 4.4 Tensors . . . 8
   - 4.5 Eigenvalues and eigenvectors . . . 9
   - 4.6 Matrix factorisation . . . 9
   - 4.7 Advanced topics . . . 9
   - 4.8 Tools and libraries . . . 9
5. Resources . . . 10
6. Summary . . . 12
7. Sources . . . 12
8. Key terms . . . 13

Where this fits: Pipeline map step 0 (Foundations). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	8 Model	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

1. Overview

Key point: Linear algebra for ML splits into eight modules: scalars, vectors, matrices, tensors, eigenvalues and eigenvectors, **matrix factorisation** (G-1177), advanced topics, and tools.

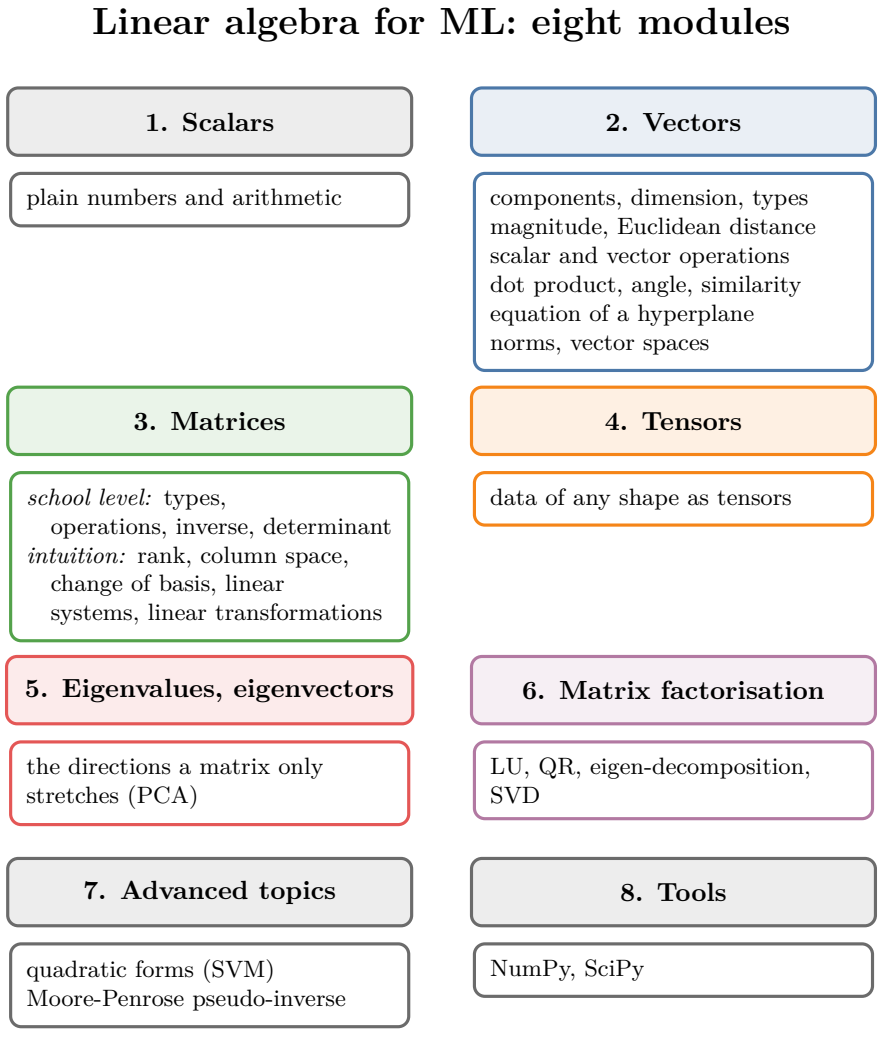


Figure 1: The eight modules of linear algebra for ML

Figure 1 shows the whole map. For each module this Note says:

- what it covers;
- why ML needs it;
- which topics can wait;
- where each topic is taught.

2. Why ML needs linear algebra

> **Key point:** Statistics and linear algebra are the two pillars of ML: statistics is how we see the data, linear algebra does the heavy lifting.

The first pillar, statistics, has its own roadmap. If statistics is the eyes of an ML engineer, linear algebra is the hands and feet: it stores the data and does every calculation on it.

Figure 2 shows the starting point on real data. Three iris flowers form a table of numbers, a **matrix** (G-1180); each row of that table is a **vector** (G-2081), an arrow from the origin in the space of the features. Every ML algorithm works on such matrices and vectors.

the data as a 3 × 2 matrix

flower	petal length	petal width
x1	1.4	0.2
x2	4.7	1.4
x3	6	2.5

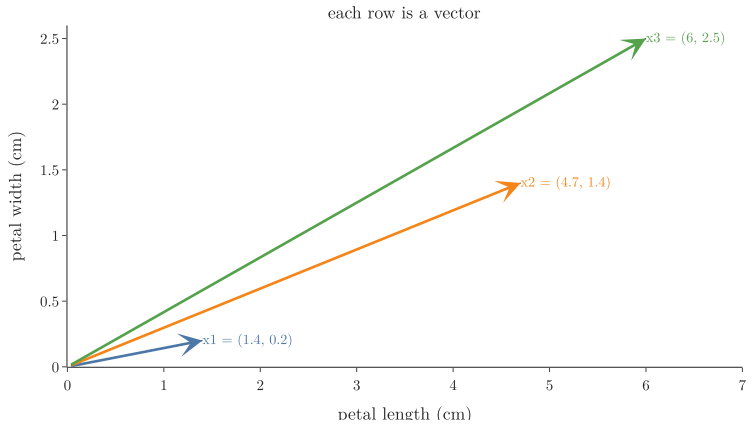


Figure 2: Three iris flowers, one per species: their petal length and width as a 3 × 2 matrix (left) and each row drawn as a vector (right).

Why ML needs it (high dimensions, data as numbers, GPU speed) is explained in why ML needs linear algebra.

2.1 A neural network is matrix maths

> **Key point:** One layer of a neural network is three steps of linear algebra: multiply a vector by a **matrix** (G-1180), add a vector, and set the negative numbers to 0.

The models do their work with the same objects. Figure 3 follows one iris flower through one layer of a **neural network** (G-1316), with numbers picked by hand so that every step can be checked:

1. **Multiply.** The flower is the **vector** (G-2081) $[0.5, 0.4]$: its petal width and its sepal width, both rescaled to lie between 0 and 1 (a raw iris sepal width is 2 to 4.4 cm, never 0.4). The layer's **weights** (G-2106) form a matrix. Multiplying the vector by the matrix gives two new numbers, each a sum of the inputs times one column of weights. In Figure 3 the weight matrix has the rows $[-2, -1]$ and $[1, 0.5]$, so, one entry per line:

$$0.5 \times (-2) = -1$$

$$0.4 \times 1 = 0.4$$

$$-1 + 0.4 = -0.6$$

$$0.5 \times (-1) = -0.5$$

$$0.4 \times 0.5 = 0.2$$

$$-0.5 + 0.2 = -0.3$$

2. **Add.** The layer's **biases** (G-284) form a second vector, added number by number. With the biases $[1.2, 0.1]$:

$$-0.6 + 1.2 = 0.6$$

$$-0.3 + 0.1 = -0.2$$

   The result is $[0.6, -0.2]$.

3. **Cut.** The **ReLU** (G-1668) function turns every negative number into 0, which leaves $[0.6, 0]$.

The next layer repeats the same three steps on $[0.6, 0]$ with its own weights and biases. A whole network is this chain of matrix products and vector additions, which the **forward propagation** (G-797) works through in full.

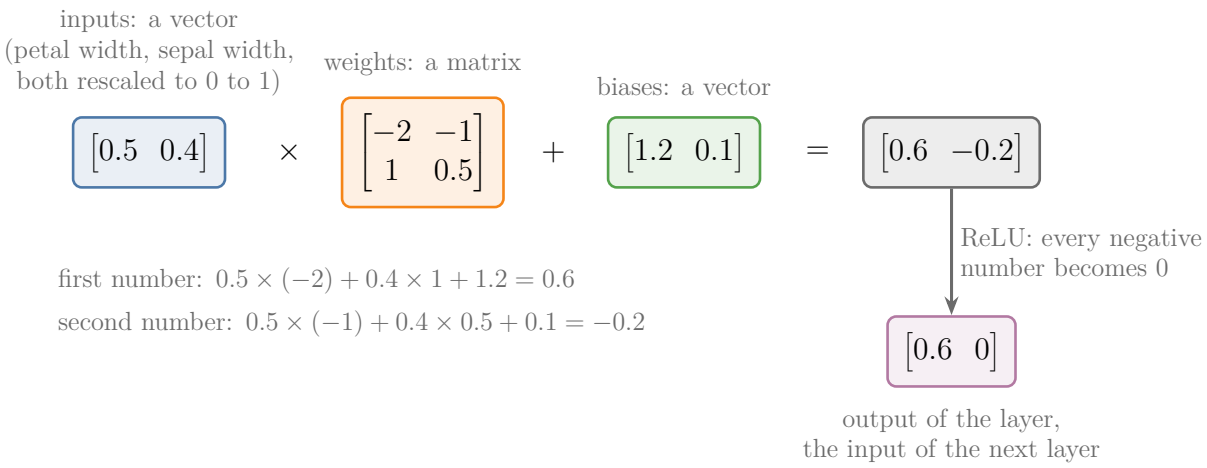


Figure 3: One layer of a neural network on one iris flower: the input vector times the weight matrix, plus the bias vector, then ReLU. Worked example after StatQuest, "Essential Matrix Algebra for Neural Networks", with our own numbers.

2.2 Two ways to know linear algebra

> **Key point:** Knowing how to compute a result and knowing what the result means are two different skills; the computer does the first, we need the second.

Every topic on the roadmap can be known in two ways:

- **Numerically:** how to carry out the operation, such as multiplying two matrices or finding an **eigenvalue** (G-665) by hand.
- **Geometrically:** what the operation does to arrows and to space, such as a **matrix** (G-1180) turning and stretching every vector.

The numeric skill carries a calculation through. The geometric skill tells us which tool fits a problem, why the tool works and how to read its result. **NumPy** (G-1369) does the numeric half for us (section 4.8), so the geometric half is the one worth our time. The maths Notes that follow therefore start each topic from its picture and then give the numbers.

3. How to read the roadmap

> **Key point:** Each topic is marked by importance, and topics needed only by a few algorithms are marked "later".

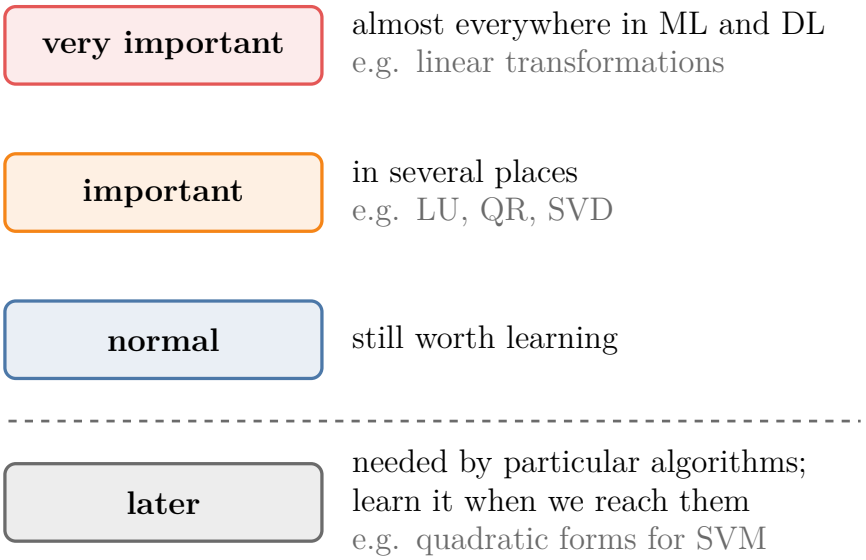


Figure 4: The marks on the roadmap: three levels of importance, and "later" for topics only particular algorithms need

Figure 4 is the key to the marks; everything above the dashed line is worth covering in one go.

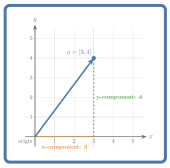
The roadmap marks every topic in three ways:

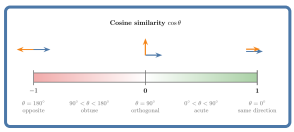
- **Importance.** A *very important* topic appears almost everywhere in ML and DL; an *important* topic appears in several places; a *normal* topic is still worth learning.
- **"Later".** A few specialised topics are needed only by particular algorithms. We can skip them now and learn them when we reach that algorithm. Most of them sit in the last modules.
- **Why it matters.** Every topic comes with a one-line description and where ML and DL use it. For example, **vectors** (G-2081) represent data points in ML, and features, weights and biases in DL.

Everything not marked "later" is worth covering in one go before moving deep into ML and DL.

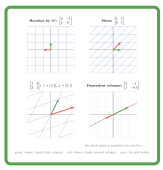
4. The eight modules

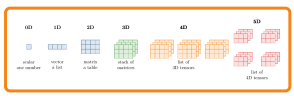
> **Key point:** Vectors and matrices are the core; **tensors** (G-1957), eigenvectors and factorisations build on them; a few advanced topics wait until an algorithm needs them.


4.2 a vector is a point

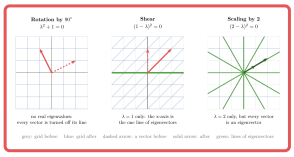


4.2 dot product, similarity


4.3 a matrix moves space



4.4 tensors of every rank



4.5 eigenvectors only stretch

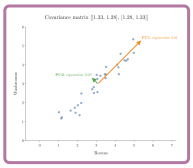

4.6 eigen-decomposition in PCA

Figure 5: A preview of six modules, each picture taken from the Note that teaches it: a vector as a point (Note MA-048), cosine similarity (Note MA-050), matrices as transformations (Note MA-053), tensors (Note ML-010), eigenvectors (Note MA-056) and the eigen-decomposition in PCA (Note ML-047)

Figure 5 shows what each module looks like once we study it; the subsection numbers match the captions.

4.1 Scalars

> **Key point:** A scalar is a plain number.

Scalars (G-1743) are ordinary numbers such as 2, 5 or -6, and the arithmetic on them. **Tensors** (G-1957) place them as 0D tensors. They need no separate study.

4.2 Vectors

> **Key point: Vectors** (G-2081) are where linear algebra starts, and where ML data starts: every observation of a dataset is a vector.

Each **observation** (G-1374; one record, one row of the data table) is a vector. This module tries to cover everything about vectors that ML uses:

Topic	Why ML needs it	Where it is taught
What vectors are, components, dimension	every data point is a vector	What a vector is
Types of vectors (row, column, ...)	data and parameters are written as rows or columns	Row vectors and column vectors

Topic	Why ML needs it	Where it is taught
Distance from origin (magnitude)	the length of a vector	Magnitude
Euclidean distance	KNN, K-means, recommender systems	Euclidean distance in KNN imputation, Euclidean distance
Operations with a scalar	shifting and scaling data, mean centring	Operations with a scalar
Operations between two vectors, dot product	similarity, projection, matrix multiplication	Computing the dot product
Equation of a line in n dimensions	linear models: linear and logistic regression, SVM	From a line to a hyperplane
Vector norms	regularisation	norm in the margin of an SVM; penalties in Ridge and Lasso
Vector spaces	the setting for everything above	a later maths Note

4.3 Matrices

> **Key point:** Matrices come in two parts: the school-level mechanics, then the intuition of what a **matrix** (G-1180) does.

The first part is the mechanics taught in school mathematics (classes 11 and 12): what matrices are, types of matrices, operations with a **scalar** (G-1743) and with a vector, the inverse and the determinant.

The second part is conceptual: the rank of a matrix, column space, change of basis, solving a system of linear equations, linear transformations, and the dot product seen as the engine of matrix multiplication. **Linear transformations** (G-1097) are marked very important.

So far, the inverse appears in the **normal equation** (G-1344) of linear regression, and the idea of a matrix as a transformation in PCA. Linear transformations are taught in what makes a transformation linear, matrix multiplication in **composition** (G-431), and the determinant in eigenvectors and eigenvalues.

4.4 Tensors

> **Key point:** Tensors are super important for deep learning, and for representing any kind of data.

What a **tensor** (G-1957) is, and how tables, text, images and videos become tensors, is taught in what a tensor is.

4.5 Eigenvalues and eigenvectors

> **Key point:** Eigenvectors are the directions a **matrix** (G-1180) only stretches; PCA is built on them.

Anyone who has studied PCA has met **eigenvalues** (G-665) and **eigenvectors** (G-666). They are taught in the eigenvectors of PCA, and in depth in eigenvectors stay on their own span.

4.6 Matrix factorisation

> **Key point:** Factorising a **matrix** (G-1180) splits it into simpler matrices; LU, QR, eigen-decomposition and SVD are the ones ML meets.

Matrix factorisation (G-1177), or **decomposition**, writes one matrix as a product of simpler ones. Four techniques are marked important: LU decomposition, QR decomposition, eigen-decomposition and SVD (singular value decomposition). Eigen-decomposition appears in the eigenvectors of PCA and the eigenbasis; SVD starts in every matrix turns a circle into an ellipse; LU and QR come in a later maths Note.

> **Extra:** Where these show up. scikit-learn's Ridge can solve its equation with SVD (`solver="svd"`, see ridge with gradient descent in scikit-learn). Least-squares problems can be solved through a QR factorisation (Trefethen and Bau 1997, Lecture 11, Algorithm 11.2), and recommender systems use SVD-like factorisations of the user-item rating matrix to predict missing ratings (Koren, Bell and Volinsky 2009).

4.7 Advanced topics

> **Key point:** Quadratic forms and the pseudo-inverse are needed only by particular algorithms, so they can wait.

- **Quadratic forms** (G-1597) are used heavily when solving SVMs (see the optimisation problem of an SVM).
- The **Moore-Penrose pseudo-inverse** (G-1262) gives an inverse for matrices that are not square, or that have no ordinary inverse. The cost of the inverse in linear regression shows why libraries need it.

Both come in a later maths Note.

4.8 Tools and libraries

> **Key point:** NumPy is how linear algebra is done in Python; SciPy adds more.

All linear algebra in the ML and DL world runs through **NumPy** (G-1369), so a strong command of it pays off everywhere. **SciPy** (G-1751), built on NumPy, adds more routines; it is used less, but we should know it exists.

Python: NumPy's linear algebra lives in `np.linalg`.

```
import numpy as np

v = np.array([3, 4])
np.linalg.norm(v)              # 5.0: length of v
A = np.array([[2, 1], [1, 3]])
np.linalg.inv(A)               # inverse of A
np.linalg.eig(A)               # eigenvalues, eigenvectors
```

Figure 6 draws what the last lines compute: the length of v is 5, and A has **eigenvalues** (G-665) 1.38 and 3.62, so it stretches its two eigenvectors by those factors without turning them.

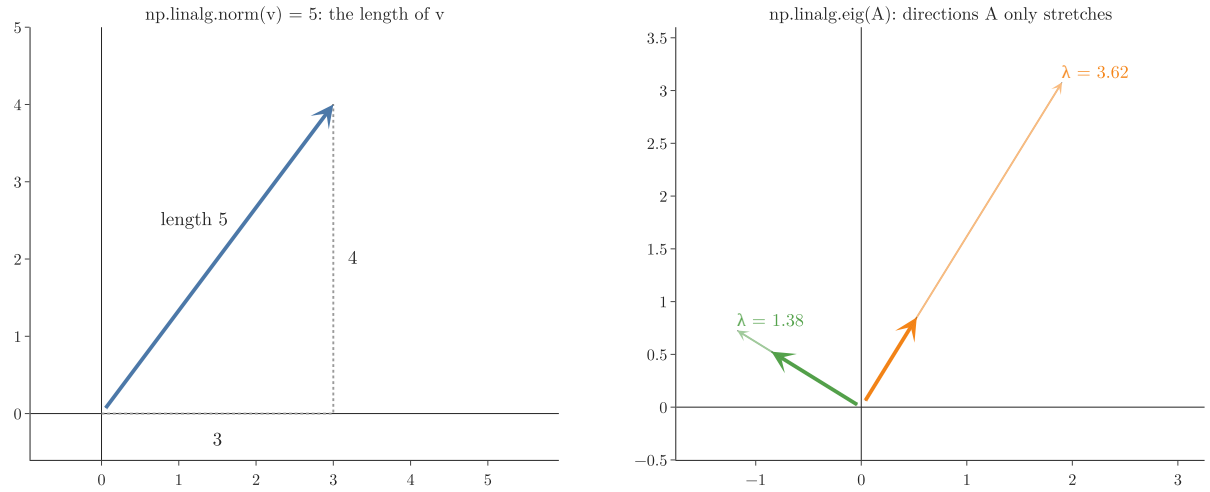


Figure 6: Left: v = (3, 4) has length 5. Right: A = [[2, 1], [1, 3]] stretches its eigenvectors (thick arrows) by its eigenvalues 1.38 and 3.62 (thin arrows), along the same lines.

5. Resources

Key point: The 3Blue1Brown series for intuition, Gilbert Strang for depth, and chapter 2 of the *Deep Learning* book as a summary.

Lesson series:

- **Sanderson, G., *Essence of Linear Algebra*, 3Blue1Brown, 3blue1brown.com.** An animated series that builds the geometric intuition for **vectors** (G-2081), matrices and transformations. Each lesson is dense, so it pays to go through it more than once.
- **Strang, G., *18.06 Linear Algebra*, MIT OpenCourseWare, ocw.mit.edu.** About 35 lectures by a renowned professor. The course covers far more than ML needs, but it is the place for depth.

Books:

- **Gilbert Strang, *Introduction to Linear Algebra*.** The book behind his lectures.
- **NCERT mathematics textbooks (classes 11 and 12).** Light on intuition, but they cover the school-level topics well.
- **Goodfellow, Bengio and Courville, *Deep Learning*, chapter 2.** A compact summary of all the linear algebra deep learning needs; worth reading twice.

Online courses:

- **Khan Academy, Linear Algebra.**
- **Coursera, Mathematics for Machine Learning: Linear Algebra** (Imperial College London).

The vectors Notes that follow this one cover the first module in depth; the rest of the **matrix** (G-1180) and eigen topics come in later maths Notes.

> **Extra:** One more free book worth knowing: *Mathematics for Machine Learning* by Deisenroth, Faisal and Ong (mml-book.github.io). Its chapters 2 to 4 (Linear Algebra, Analytic Geometry, Matrix Decompositions) cover the vectors, matrices, **eigenvalues** (G-665) and factorisations of this roadmap, including SVD in section 4.5.

6. Summary

Module	What it covers	Status	Why it matters
Scalars	plain numbers	no study needed	they are 0D tensors, nothing new to learn
Vectors	components, distance, operations, dot product, hyperplanes, norms	core	every data point is a vector
Matrices	mechanics, then rank, spaces, linear transformations	core	a dataset is a matrix, and a network layer is a matrix product
Tensors	data of any shape	core for DL	tables, text, images and videos all become tensors
Eigenvalues and eigenvectors	directions a matrix only stretches	needed for PCA	PCA is built on them
Matrix factorisation	LU, QR, eigen-decomposition, SVD	important	Ridge, least squares and recommender systems solve through them
Advanced topics	quadratic forms, pseudo-inverse	later	only particular algorithms (SVM, linear regression solvers) need them
Tools	NumPy, SciPy	throughout	all linear algebra in ML and DL runs through NumPy

- Statistics and linear algebra are the two pillars of ML and DL: statistics is how we see the data, and linear algebra stores it and does every calculation on it.
- Learn the unmarked topics in one go, because they appear almost everywhere in ML and DL; leave the "later" topics until an algorithm needs them, because only that algorithm uses them.
- For every topic, know where ML uses it, so each "later" topic can be picked up when its algorithm arrives.
- Learn each topic's picture as well as its numbers, because **NumPy** (G-1369) does the numbers and the picture tells which tool fits a problem.
- So the roadmap answers, module by module, which linear algebra ML needs now and which can wait.

7. Sources

Built from

- CampusX, "Linear Algebra Roadmap for Machine Learning and Deep Learning | Complete Guide | CampusX", YouTube, https://www.youtube.com/watch?v=rIsCKVyh4dI
- Sanderson, G. (3Blue1Brown), "Essence of linear algebra preview", YouTube, https://www.youtube.com/watch?v=kjBOesZCoqc
- Starmer, J. (StatQuest), "Essential Matrix Algebra for Neural Networks, Clearly Explained!!!", YouTube, https://www.youtube.com/watch?v=ZTt9gsGcdDo

Other references

- Trefethen, L. N. and Bau, D. (1997). *Numerical Linear Algebra.* SIAM. Lecture 11, "Least squares problems", Algorithm 11.2.
- Koren, Y., Bell, R. and Volinsky, C. (2009). "Matrix factorization techniques for recommender systems." *IEEE Computer* 42(8), 30–37.
- Deisenroth, M. P., Faisal, A. A. and Ong, C. S. (2020). *Mathematics for Machine Learning.* Cambridge University Press. Chapters 2–4; §4.5.

8. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Matrix factorisation (decomposition) (G-1177)	Writing a matrix as a product of simpler matrices.
NumPy (G-1369)	Python's library for arrays and linear algebra.
SciPy (G-1751)	A Python library built on NumPy that adds more scientific routines, such as extra linear algebra and statistics functions.
Matrix (G-1180)	A table of numbers in rows and columns (a 2D tensor); it can hold a dataset (one row per observation), and multiplying a vector by it transforms the vector.
Vector (G-2081)	A list of numbers: a 1D tensor.
Neural network (G-1316)	A model made of many simple connected units (neurons) in layers, loosely inspired by the brain; it learns the weights of its connections from data and is the model deep learning uses.
Weight (in a network) (G-2106)	A number the network learns on the connection between two neurons; it sets how strongly one neuron's output counts in the next neuron's sum.
Bias (of a perceptron) (G-284)	The weight on a constant input of 1; it shifts the boundary away from the origin.

Term	Meaning
ReLU (G-1668)	Rectified linear unit, the activation $\max(0, z)$: it passes a positive input unchanged and turns a negative one into 0; the usual default activation for hidden layers.
Forward propagation (G-797)	Passing one row of inputs through the network, layer by layer, to get the prediction.
Eigenvalue (G-665)	The number by which a matrix stretches or shrinks one of its eigenvectors: $A\mathbf{v} = \lambda\mathbf{v}$. In PCA, the eigenvalues of the covariance matrix are the variances along the principal components.
Tensor (G-1957)	A container of numbers arranged along one or more axes.
Scalar (G-1743)	A single number, such as 5 or -6: a 0D tensor; multiplying a vector by it scales the vector, which is where the name comes from.
Observation (G-1374)	One record of a dataset: one row of the data table.
Linear transformation (G-1097)	A change of the whole plane by a matrix that keeps grid lines straight and evenly spaced, and the origin fixed; algebraically, $L(\mathbf{v}+\mathbf{w}) = L(\mathbf{v}) + L(\mathbf{w})$ and $L(c\mathbf{v}) = cL(\mathbf{v})$.
Normal equation (G-1344)	A formula that gives all linear-regression coefficients in one step, with no iterations (a closed-form solution): $\beta = (X^\mathsf{T}X)^{-1}X^\mathsf{T}y$.
Composition (G-431)	Applying one transformation and then another, seen as one overall transformation.
Eigenvector (G-666)	A non-zero vector that a matrix only stretches or shrinks, without turning it off its own line: $A\mathbf{v} = \lambda\mathbf{v}$. In PCA, the eigenvectors of the covariance matrix are the directions of the principal components.
Quadratic form (G-1597)	An expression such as $\mathbf{x}^\mathsf{T}A\mathbf{x}$ that turns a vector into one number made only of squared entries and products of pairs of entries, such as $x_1^2 + 3x_2^2$; it is the curved part of a quadratic program's objective.
Moore–Penrose pseudo-inverse (A^+) (G-1262)	The closest thing to an inverse for a matrix that is not square or has no inverse (a generalised inverse): the SVD inverted with zero singular values left at zero, $V\Sigma^+U^\mathsf{T}$.

Term	Meaning
Singular value decomposition (SVD) (G-1813)	A way to write any matrix, square or not, as three simple steps: turn, stretch by the singular values, turn ($A = U\Sigma V^{\mathsf{T}}$: orthogonal, diagonal, orthogonal; a factorisation). ML uses it for PCA and to compress data.